Princeton University

ORF 363/COS 323
Final Exam, Fall 2021

December 15, 2021

Instructor:
A.A. Ahmadi

AIs:
Becker, Chaudhry, Dibek, Silin, Yu

1. Please write your name on the first page of your solutions. Next to it, please write out and sign the following pledge: "I pledge my honor that I have not violated the Honor Code or the rules specified by the instructor during this examination."
2. The exam is not to be discussed with *anyone* except possibly the instructors and the AIs. You can only ask *clarification questions*, and only as *public* (and preferably non-anonymous) questions on Ed Discussion. No emails.
3. You are allowed to consult the lecture notes, your own notes, the reference books of the course as indicated on the syllabus, the problem sets and their solutions (yours and ours), the midterm and its solutions (yours and ours), the practice midterm and final exams and their solutions, all Ed Discussion posts, all videos, but *nothing else*. You can only use the Internet in case you run into problems related to software.
4. You may refer to facts proven in the notes or problem sets without reproving them.
5. For all computational problems, include your code. The output that you present should come from your code.
6. Unless you have been granted an extension because of overlapping finals, the exam is to be turned in on Friday (December 17, 2021) at 10 AM EST on Gradescope as a single PDF file. You are free to write your solutions on paper or on a tablet, or to type them up. Only the latest version submitted before your deadline will be graded.
7. Each question has 25 points. You need to justify your answers to receive full credit.

Problem 1: Friends in High Places

You follow the path of Jeff Bezos and become a billionaire after graduation. Naturally, you feel compelled to send your friends to space. You would like to send as many friends as possible, but on a safety-critical mission like this, there is no room for disagreement on the spaceship. If you have n friends to choose from, you write down an $n \times n$ matrix describing the disagreement between any pair of friends. The entries of this matrix are real numbers between 0 and 1, where 0 means "they get along perfectly" and 1 means "they fight all the time". This matrix is symmetric and has zero diagonal. Here is an example of a disagreement matrix when there are five friends to choose from:

$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} 0 & 0.1 & 0.6 & 0.4 & 0.7 \\ 0.1 & 0 & 0.4 & 0.2 & 1.0 \\ 0.6 & 0.4 & 0 & 0.3 & 0.4 \\ 0.4 & 0.2 & 0.3 & 0 & 0.7 \\ 0.7 & 1.0 & 0.4 & 0.7 & 0 \end{bmatrix} \end{array}.$$

For a given subset of friends, the total disagreement is the sum of all disagreement values between pairs in the subset. For example, the set of friends $\{1, 3, 4\}$ incurs a total disagreement of $0.6 + 0.4 + 0.3 = 1.3$. Given your disagreement matrix M and a nonnegative scalar p, you want to find the maximum number of friends you can send to space with total disagreement at most p.[1] You formulate this problem as follows:

$$\begin{aligned} \eta^* \quad := \quad & \max_{x \in \mathbb{R}^n} \quad \sum_{i=1}^{n} x_i \\ & \text{s.t.} \quad \sum_{i,j=1}^{n} M_{ij} x_i x_j \leq 2p \\ & \qquad\quad x_i(x_i - 1) = 0 \ \text{ for } i = 1, \ldots, n. \end{aligned} \tag{1}$$

1. Consider the following linear program[2]:

$$\begin{aligned} \eta^*_{LP} \quad := \quad & \max_{x \in \mathbb{R}^n} \quad \sum_{i=1}^{n} x_i \\ & \text{s.t.} \quad \sum_{i,j=1}^{n} M_{ij} \max\{x_i + x_j - 1, 0\} \leq 2p \\ & \qquad\quad 0 \leq x_i \leq 1 \ \text{ for } i = 1, \ldots, n. \end{aligned} \tag{2}$$

   Show that $\eta^* \leq \eta^*_{LP}$.

[1] While you are not required to show this, the problem you are facing is NP-hard.

[2] Even though no points are assigned to this, you may want to convince yourself that this optimization problem can indeed be reformulated as a linear program.

2. Let S^{n+1} denote the set of symmetric $(n+1)\times(n+1)$ matices. Let $\hat{M} \in S^{n+1}$ denote the matrix obtained from M by adding a row and a column of zeros, i.e., M is the upper left $n \times n$ submatrix of $\hat{M}$ and the other entries of $\hat{M}$ are all zeros. For $i = 1, \ldots, n$, let $A^i \in S^{n+1}$ denote the matrix with $A^i_{ii} = 1$, $A^i_{i,n+1} = A^i_{n+1,i} = -0.5$, and zeros everywhere else. Consider the following semidefinite program:

$$\begin{aligned} \eta^*_{SDP} \quad := \quad & \max_{X \in S^{n+1}} && \mathrm{Tr}(X) - 1 \\ & \text{s.t.} && \mathrm{Tr}(\hat{M}X) \le 2p \\ & && \mathrm{Tr}(A^i X) = 0 \text{ for } i = 1, \ldots, n \\ & && X_{n+1,n+1} = 1 \\ & && X \succeq 0. \end{aligned} \tag{3}$$

Show that $\eta^* \le \eta^*_{SDP}$.

3. Solve problems (2) and (3) for the 14×14 matrix M given in `disagreement_matrix.mat` and $p = 1.12$. You may use the following code to load the disagreement matrix:

```
load('disagreement_matrix.mat')
```

Report the two optimal values and specify in each case whether they are equal to η^*. Which of your 14 friends should you be sending to space?

Solution:

1. Problems (1) and (2) have the same objective function. Let $x \in \mathbb{R}^n$ be feasible for (1). Then, $\sum_{i,j=1}^n M_{ij}x_ix_j \le 2p$ and $x_i \in \{0,1\}$ for $i = 1, \ldots, n$. We show that x is feasible for (2). Observe that $\max\{x_i+x_j-1, 0\} = 1$ if $x_i = x_j = 1$, and 0 otherwise. Therefore, $\sum_{i,j=1}^n M_{ij}\max\{x_i + x_j - 1, 0\} \le 2p$ is satisfied and x is feasible for (2). Since any feasible solution to (1) is also feasible for (2), we have $\eta^* \le \eta^*_{LP}$.

2. Note that the objective function in (1) can be replaced with $\sum_{i=1}^n x_i^2$ since $x_i \in \{0,1\}$ for $i = 1, \ldots, n$. Note also that $\sum_{i,j=1}^n M_{ij}x_ix_j \le 2p$ can be rewritten as $\mathrm{Tr}(Mxx^T) \le 2p$. Now, let $\hat{x} = (x_1, \ldots, x_n, 1)$ and introduce a new variable $X = \hat{x}\hat{x}^T$. We then have $X_{n+1,n+1} = 1$ and the objective function $\sum_{i=1}^n x_i^2$ can be written as $\mathrm{Tr}(X) - 1$. As the last row and column of $\hat{M}$ consist of zeros, the constraint $\mathrm{Tr}(Mxx^T) \le 2p$ can be rewritten as $\mathrm{Tr}(\hat{M}X) \le 2p$. Also, for $i = 1, \ldots, n$, the constraint $x_i(x_i - 1) = 0$ can be rewritten as $\mathrm{Tr}(A^iX) = 0$. Moreover, since $X = \hat{x}\hat{x}^T$, we have $X \succeq 0$. Hence, for any feasible solution x to (1), the matrix $X = \hat{x}\hat{x}^T$ is feasible to (3) with the same objective value. Therefore, we have $\eta^* \le \eta^*_{SDP}$.

3. We solve problems (2) and (3) for the matrix M given in `disagreement_matrix.mat` and $p = 1.12$ using the code below:

```
load('disagreement_matrix.mat')

[n,~] = size(M);
p = 1.12;

% LP relaxation

total_sum = 0;

cvx_begin quiet
variable x(n,1)
maximize(sum(x))

for i=1:n
    for j=1:n
        total_sum = total_sum + M(i,j)*max(x(i)+x(j)-1,0);
    end
end

total_sum <= 2*p

0 <= x
x <= 1

cvx_end

% SDP relaxation

Mhat = zeros(n+1, n+1);
Mhat(1:n,1:n) = M;

A = zeros(n+1,n+1,n);

for i=1:n
    A(i,i,i) = 1;
    A(i,n+1,i) = -0.5;
    A(n+1,i,i) = -0.5;
end

cvx_begin sdp quiet
variable X(n+1, n+1) symmetric
maximize(trace(X)-1)

trace(Mhat*X) <= 2*p

for i=1:n
    trace(A(:,:,i)*X) == 0
end

X(n+1, n+1) == 1
```

```
X >= 0

cvx_end

%recover a solution
X = round(X);

X(:,end) = []; %Delete the last column
X(end,:) = []; %Delete the last row

[V,D] = eig(X);

z1 = V(:,n);
z1 = round(z1/z1(1));

find(z1)
```

The LP optimal value is $\eta^*_{LP} = 7.1827$ and the SDP optimal value is $\eta^*_{SDP} = 5$. Hence, the LP bound is not tight. The optimal solution X^* of the SDP is obtained as:

$$X^* = \begin{pmatrix} 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

Similar to problem set 8, we compute the eigenvector corresponding to the nonzero eigenvalue of the optimal matrix. After deleting the last row and column of X^*, the eigenvector that correspond to the unique nonzero eigenvalue is given as:

$$v = \begin{pmatrix} 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

An optimal set F of friends you should send to space is obtained by the indices of the entries that are 1 in v, that is, the vector v corresponds to the indicator vector of F:

$$F = \{1, 4, 5, 7, 9\}.$$

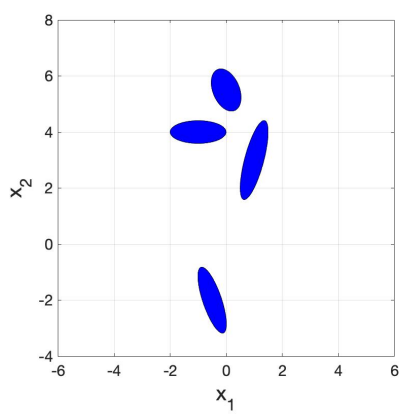
Problem 2: Winter Break Trip

For the winter break, you are planning to travel to a national park with 4 lakes. Each lake L_i has the shape of an ellipse with known center $c_i \in \mathbb{R}^2$ and shape matrix $A_i \in \mathbb{R}^{2\times 2}$, $A_i \succ 0$:

$$L_i = \left\{ x \in \mathbb{R}^2 \mid (x - c_i)^\top A_i (x - c_i) \leq 1 \right\}, \quad i = 1, \ldots, 4.$$

The lakes are shown in the figure below and correspond to the following data:

$$A_1 = \begin{bmatrix} 1 & 0 \\ 0 & 6 \end{bmatrix}, \quad A_2 = \begin{bmatrix} 8 & -2 \\ -2 & 1 \end{bmatrix}, \quad A_3 = \begin{bmatrix} 4 & 1 \\ 1 & 2 \end{bmatrix}, \quad A_4 = \begin{bmatrix} 8 & 2.5 \\ 2.5 & 1.5 \end{bmatrix},$$
$$c_1 = [-1, 4]^\top, \quad c_2 = [1, 3]^\top, \quad c_3 = [0, 5.5]^\top, \quad c_4 = [-0.5, -2]^\top.$$



You want to visit all 4 lakes, one lake per day. You need to pick a permanent location $z \in \mathbb{R}^2$ for your tent where you will be staying overnight. On the i-th day, you will be hiking from your tent to the i-th lake and back (using the shortest route to the closest point of each lake). For simplicity, no restrictions are imposed on the permanent location z of your tent. Write down and solve two convex optimization problems which would respectively minimize:

(a) the sum of squared Euclidean distances to all the lakes;

(b) the maximum Euclidean distance to any of the lakes (i.e. on no day you want to hike too much).

Report the optimal solutions. You can optionally plot your optimal solutions on the MATLAB figure provided (no points are assigned to plotting). For instance, to plot the point $[2, 1]^\top$ on the map, you can use the following code:

```
openfig('lakes.fig')
plot(2, 1, 'k*', 'MarkerSize', 10, 'LineWidth', 2)
```

Solution:

(a) We introduce variables $x^{(i)} \in \mathbb{R}^2$, $i = 1, \ldots, 4$, for the closest locations of the lakes from z. The convex optimization problem then is:

$$\begin{aligned} \min_{z, x^{(1)}, \ldots, x^{(4)} \in \mathbb{R}^2} \quad & \sum_{i=1}^{4} \|z - x^{(i)}\|_2^2 \\ \text{s. t.} \quad & (x^{(i)} - c_i)^\top A_i (x^{(i)} - c_i) \leq 1 \quad \text{for all } i = 1, \ldots, 4. \end{aligned}$$

The objective is convex as sum of convex quadratic functions (squared Euclidean norm, so the Hessian w.r.t. its argument is twice the identity matrix) of linear transforms of the variables (alternatively one can check that the overall Hessian is PSD). The constraints are convex as in the LHS we have quadratic functions with PSD Hessians, and in the RHS we have just the constant. Hence, this is a convex optimization problem.

(b) Similarly, the convex optimization problem is:

$$\begin{aligned} \min_{z, x^{(1)}, \ldots, x^{(4)} \in \mathbb{R}^2} \quad & \max_{i=1,\ldots,4} \|z - x^{(i)}\|_2^2 \\ \text{s. t.} \quad & (x^{(i)} - c_i)^\top A_i (x^{(i)} - c_i) \leq 1 \quad \text{for all } i = 1, \ldots, 4. \end{aligned}$$

The objective is convex as max of convex functions (their convexity was discussed in (a)). The constraints are convex as was argued in (a). Hence, this is a convex optimization problem.

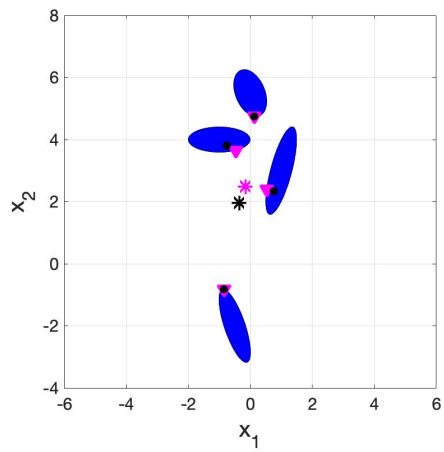
For convenience, using the trick with introducing new variable to get rid of the maximization in the objective, we can rewrite the convex optimization problem in the following form:

$$\begin{aligned} \min_{\substack{z, x^{(1)}, \ldots, x^{(4)} \in \mathbb{R}^2 \\ b \in \mathbb{R}}} \quad & b \\ \text{s. t.} \quad & \|z - x^{(i)}\|_2^2 \leq b \quad \text{for all } i = 1, \ldots, 4, \\ & (x^{(i)} - c_i)^\top A_i (x^{(i)} - c_i) \leq 1 \quad \text{for all } i = 1, \ldots, 4. \end{aligned}$$

The code for both parts is:

```
clear all
A = zeros(2,2,4);
c = zeros(2,4);
```

```
A(:,:,1) = [1 0; 0 6];
c(:,1) = [-1; 4];

A(:,:,2) = [8 -2; -2 1];
c(:,2) = [1; 3];

A(:,:,3) = [4 1; 1 2];
c(:,3) = [0; 5.5];

A(:,:,4) = [8 2.5; 2.5 1.5];
c(:,4) = [-0.5; -2];
openfig('lakes.fig');

cvx_begin
    variables z(2,1) x(2,4)
    minimize (pow_pos(norm(z-x(:,1), 2), 2) ...
              + pow_pos(norm(z-x(:,2), 2), 2) ...
              + pow_pos(norm(z-x(:,3), 2), 2) ...
              + pow_pos(norm(z-x(:,4), 2), 2))
    subject to
        for i = 1:4
            (x(:,i)-c(:,i))'*A(:,:,i)*(x(:,i)-c(:,i)) <= 1;
        end
cvx_end
z1 = z
plot(z1(1), z1(2), 'm*', 'MarkerSize', 10, 'LineWidth', 2);

cvx_begin
    variables z(2,1) x(2,4) b
    minimize (b)
    subject to
        for i = 1:4
            pow_pos(norm(z-x(:,i), 2), 2) <= b;
            (x(:,i)-c(:,i))'*A(:,:,i)*(x(:,i)-c(:,i)) <= 1;
```




```
        end
cvx_end
z2 = z
plot(z2(1), z2(2), 'k*', 'MarkerSize', 10, 'LineWidth', 2);

z1
z2
```

Optimal solution for (a) is $z^* = [-0.1581,\ 2.4960]^\top$ (denoted as a magenta star in the figure; magenta triangles on the borders of the lakes are the closest points $x^{(i)}$'s). Optimal solution for (b) is $z^* = [-0.3605,\ 1.9636]^\top$ (denoted as a black star in the figure; black circles are $x^{(i)}$'s, some of them are not necessarily on the border according to our convex problem formulation).

You can also see HW3 Problem 3 for related ideas.

Problem 3: Dynamical Leontief Economy

Recall the Leontief input-output model from Lecture 10. Imagine an economy where each sector decides its production amount for tomorrow based on the demand today. Let n denote the number of sectors, $C \in \mathbb{R}^{n \times n}$ be the consumption matrix, $d \in \mathbb{R}^n$ be the demand vector of the open sector (e.g., the general public), and $x_t \in \mathbb{R}^n$ be the vector of production on day t. Then, x_t evolves according to the dynamics:

$$x_{t+1} = Cx_t + d.$$

As we have seen, the right amount of production in the Leontief model is $x^* = (I - C)^{-1}d$. If the initial production $x_0 \in \mathbb{R}^n$ is away from x^*, one might wonder if the economy can balance itself given enough time. More precisely, we want to see if $x_t \to x^*$ as $t \to \infty$. Consider the same Leontief economy as in Problem Set 6. There are four sectors and a consumption matrix C given by:

	Agriculture	Energy	Transportation	Manufacturing
Agriculture	0.1	0	0.15	0.15
Energy	0.15	0.12	0	0
Transportation	0.1	0.15	0.2	0.15
Manufacturing	0	0.1	0.1	0.7.

Prove that (no matter what the vector d is) for any initial $x_0 \in \mathbb{R}^4$, we have $x_t \to x^*$.
Hint: Show this by presenting an appropriate quadratic Lyapunov function. It may be helpful to perform a change of variables first.

Solution:

We wish to show $x_t \to x^*$. Another way to say this is $x_t - x^* \to 0$. Define the new variables $y_t = x_t - x^*$. We can then derive dynamics for y_t:

$$\begin{aligned}
y_{t+1} &= x_{t+1} - x^* \\
&= Cx_t + d - x^* \\
&= C(y_t + x^*) + d - x^* \\
&= Cy_t + Cx^* + d - x^* \\
&= Cy_t + (C - I)x^* + d \\
&= Cy_t - (I - C)(I - C)^{-1}d + d \\
&= Cy_t - d + d
\end{aligned}$$

$$= Cy_t$$

Now it suffices to show that $y_t \to 0$; we can use a Lyapunov function to show this. Let's use the following Lyapunov function: $V(y) = ||y||^2 = y^T y$. To apply the stability theorem we must check four conditions.

(i) $V(0) = 0$.

(ii) $V(y) > 0, \ \forall y \neq 0$.

(iii) $V(y) \to \infty$ as $||y|| \to \infty$.

(iv) $V(Cy) < V(y), \ \forall y \neq 0$.

The first three conditions follow from the fact that we have chosen the square of a norm as our Lyapunov function. To check the last condition, observe:

$$V(y) - V(Cy) = y^T y - (Cy)^T (Cy) = y^T y - y^T C^T C y = y^T (I - C^T C) y.$$

We can verify that this quantity is positive (and therefore $V(Cy) < V(y)$) for any nonzero vector y if the matrix $I - C^T C$ is positive definite. In this case we have

$$I - C^T C = \begin{bmatrix} 0.9575 & -0.0330 & -0.0350 & -0.0300 \\ -0.0330 & 0.9531 & -0.0400 & -0.0925 \\ -0.0350 & -0.0400 & 0.9275 & -0.1225 \\ -0.0300 & -0.0925 & -0.1225 & 0.4650 \end{bmatrix}.$$

One can easily check that this matrix is positive definite, for instance, by putting it into MATLAB and observing that all of its eigenvalues are positive.
Alternatively, one might want to search for a Lyapunov function instead of guessing one. Let us use a quadratic Lyapunov function $V(x) = x^T P x$. It can be checked that the first conditions are implied if we have $P - I \succeq 0$. The fourth condition is implied if we have $P - C^T P C - I \succeq 0$. We can search for such a matrix P with the following code:

```
cvx_begin
variable P(n,n) symmetric
P - eye(n) == semidefinite(n);
P - C'*P*C - eye(n) == semidefinite(n);
cvx_end
```

By using CVX we can check that this program is not infeasible and we can find a suitable matrix P for the result we seek.

Problem 4: Making The Most Out of an Optimal Value Blackbox

Consider the following optimization problem

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & x \in \Omega, \end{aligned} \tag{4}$$

where $f : \mathbb{R}^n \to \mathbb{R}$ is a continuous function and $\Omega \subseteq \mathbb{R}^n$ is a nonempty, closed, and bounded set. Suppose you have a blackbox that can return the optimal value of any optimization problem with a continuous objective function and a nonempty, closed, and bounded feasible set. How can you use only this blackbox (potentially many times) to:

(a) find an optimal solution to (4),

(b) test if the optimal solution to (4) is unique.

In each case, how many calls to the blackbox does your approach require?

Solution:

(a) First note that an optimal solution exists by Weierstrass theorem. Then we find the optimal value:

$$\begin{aligned} c^* := \min \quad & f(x) \\ \text{s.t.} \quad & x \in \Omega. \end{aligned}$$

Next we will be constructing an optimal solution $x^* = [x_1^*, \ldots, x_n^*]^\top$ component by component. The first component can be found as the optimal value of the following optimization problem (with continuous objective and nonempty, closed, and bounded feasible set):

$$\begin{aligned} x_1^* := \min \quad & x_1 \\ \text{s.t.} \quad & x \in \Omega, \\ & f(x) = c^*. \end{aligned}$$

The idea is that the newly added constraint $f(x) = c^*$ restricts the feasible set to the set of optimal solutions, and the objective allows to pick a solution with the smallest first component, returning it as the optimal value. We consequently repeat the process on each step fixing previously obtained components of the optimal solution. For $i = 2, \ldots, n$

$$\begin{aligned} x_i^* := \min \quad & x_i \\ \text{s.t.} \quad & x \in \Omega, \\ & f(x) = c^*, \\ & x_k = x_k^* \quad \text{for all} \;\; k = 1, \ldots, i-1. \end{aligned}$$

(Again, the objectives are continuous, and the feasible sets are nonempty, closed, and bounded.) The approach requires $n+1$ calls to the blackbox.

(b) Obtain the optimal value c^* as in (a). Then consider the following optimization problem (with continuous objective and nonempty, closed, and bounded feasible set)

$$\begin{aligned} \min \quad & -\|x-y\| \\ \text{s.t.} \quad & x, y \in \Omega, \\ & f(x) = c^*, \\ & f(y) = c^*. \end{aligned}$$

Clearly, the optimal value is 0 if and only if the solution to the original optimization problem is unique. The idea is somewhat similar to HW1 Problem 1.2 (Modelling Sudoku). The approach requires 2 calls to the blackbox.